

BLAST MAZE

ARENA BOMBER

MANUAL TÉCNICO

Arquitectura, módulos, algoritmos, API, pruebas y mantenimiento

Versión	1.1
Fecha	24 de septiembre de 2026
Integrantes	David Domínguez, Yenni Lopez y Samuel Pantaleon
Plataforma	Navegador web · HTML5 Canvas · Node.js

Documento de entrega académica y referencia del proyecto.

Contenido

Sección	Incluye
1. Descripción técnica	Objetivo, alcance y tecnologías.
2. Arquitectura y pantallas	Flujo de ejecución e interfaz.
3. Organización y desarrollo	Archivos, dependencias y construcción por etapas.
4. Sistemas del juego	Mapa, colisiones, bombas, enemigos, mejoras y puntuación.
5. Persistencia	API REST, validación y formato de datos.
6. Instalación	Preparación y ejecución local.
7. Pruebas y resultados	Casos de comprobación y evidencia disponible.
8. Mantenimiento y recursos	Parámetros, sprite sheets, audio y procedencia.
9. Diagnóstico y cierre	Problemas, soluciones y conclusiones.

1. Descripción técnica

Blast Maze: Arena Bomber es una aplicación web 2D construida con JavaScript sin frameworks. Renderiza el juego en HTML5 Canvas, mantiene la interfaz y menús en HTML/CSS y usa un servidor Node.js basado en módulos nativos para persistir estadísticas en JSON.

Objetivo: completar niveles de un laberinto generado automáticamente mediante la destrucción de obstáculos, la eliminación de enemigos y el acceso a una salida oculta, mientras se acumula puntuación y se consultan resultados persistentes. El alcance de este documento es el código y las capturas disponibles del proyecto; los resultados de pruebas en el equipo final se deben confirmar antes de entregar.

1.1 Alcance

- Generación procedural de laberintos y progresión indefinida.
- Bombas normales y especiales, explosiones en cruz y reacciones en cadena.
- Cuatro comportamientos de enemigos: aleatorio, BFS, A* y predictivo.
- Ocho tipos de power-up y un inventario visible.
- Interfaz espacial adaptable, audio, pausa, depuración y resultados.
- API local para registrar y consultar estadísticas de partidas.

1.2 Tecnologías

Capa	Tecnología	Responsabilidad
Presentación	HTML5 + CSS3	Estructura, paneles, menús, escalado y tema espacial.
Renderizado	Canvas 2D	Mapa, sprites, animaciones, explosiones, rutas y hitboxes.
Lógica	JavaScript ES6	Estado, clases, entrada, IA, colisiones y bucle principal.
Audio	HTMLAudioElement	Música de menú/partida y efectos WAV.
Servidor	Node.js (http, fs, path)	API REST y persistencia en archivo JSON.
Datos	JSON	Historial de hasta 1000 partidas.

2. Arquitectura

Componente	Entradas	Salidas / colaboración
index.html	Acciones del usuario	Carga estilos y scripts; aloja Canvas y pantallas.
styles.css	Clases y estados de interfaz	Diseño adaptable, visibilidad y ambientación.
main.js	Teclado, tiempo, objetos del juego	Coordina bucle, HUD, niveles, audio y API.
Clases de dominio	Mapa, deltaTime, jugador	Actualizan y dibujan jugador, bombas, enemigos y mejoras.
server.js	GET/POST HTTP	Lee y escribe server/data/scores.json.
Assets	PNG, MP3 y WAV	Sprites, marco espacial, música y efectos.

Bucle principal requestAnimationFrame calcula deltaTime. Si la partida está activa, actualiza temporizador, jugador, bombas, power-ups, enemigos, colisiones y salida; después dibuja el fotograma completo.

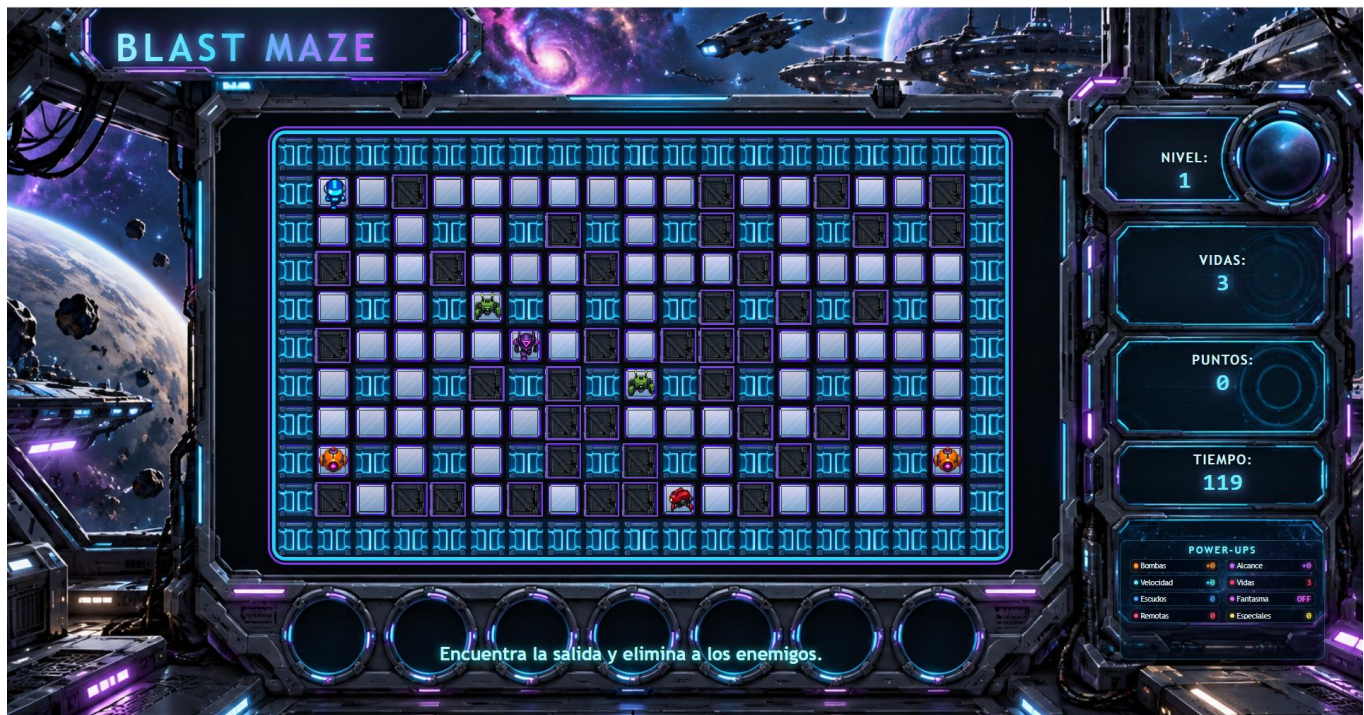
2.1 Flujo de estado

Estado	Activación	Comportamiento
Menú	Carga o regreso desde pausa	Juego detenido; permite iniciar, ver información/resultados y audio.
Jugando	Comenzar o continuar	Se actualizan entidades y temporizador.

Estado	Activación	Comportamiento
Pausa	P o pérdida de foco	Se congelan lógica y tiempo.
Nivel completado	Jugador entra al portal habilitado	Se otorga bono y espera confirmación.
Fin	Vidas = 0 o cierre definido	Guarda estadísticas y muestra resumen.

2.2 Diseño de las pantallas

Pantalla	Elementos y propósito
Menú principal	Título, controles, comenzar, instrucciones y créditos, resultados y audio.
Partida	Canvas escalable, panel lateral de nivel, vidas, puntos y tiempo, inventario y mensajes.
Instrucciones	Objetivo, movimiento, bombas, power-ups, enemigos y créditos.
Pausa	Permite continuar o regresar al menú principal; detiene el temporizador.
Nivel completado	Muestra la bonificación y botón de siguiente nivel.
Fin del juego	Resume puntuación final y ofrece jugar de nuevo.
Resultados	Cuatro indicadores y tabla de partidas consultadas a Node.js.



Captura de una versión del escenario espacial; verificar su correspondencia con la versión final antes de entregar.

3. Organización del proyecto

Ruta	Contenido
index.html	Documento principal y pantallas superpuestas.
css/styles.css	Tema espacial, HUD, menús, resultados y responsive.
js/main.js	Orquestación del juego y comunicación con la API.
js/map.js	MazeGenerator y generateLevel.
js/player.js	Movimiento, colisión, estados y dibujo.
js/bomb.js	Mecha, explosión, alcance y bloques destruidos.
js/enemy*.js	Enemigo base/aleatorio, BFS, A* y predictivo.

Ruta	Contenido
js/powerup.js	Tipos, recolección, iconos y aura.
js/audio.js	Música, efectos, silencio y cambio de escena.
assets/sprites/	Sprites y hojas de animación.
assets/audio/	Música MP3 y efectos WAV.
assets/ui/space_frame.png	Marco e interfaz espacial.
server/server.js	Servicio HTTP local.
server/data/scores.json	Persistencia de resultados.

3.1 Orden de carga

El index.html revisado carga: map.js, player.js, bomb.js, enemy.js, enemyBFS.js, enemyAStar.js, enemyPredictive.js, powerup.js, main.js y audio.js. map.js y las clases deben estar disponibles antes de que main.js ejecute generateLevel(1) y cree las entidades. Un cambio de orden puede producir ReferenceError o detener el dibujo del mapa.

3.2 Desarrollo documentado por etapas

1. Definir requisitos: juego en Canvas, cuatro tipos de IA, bombas, mejoras y estadísticas en Node.js.
2. Diseñar el DOM de index.html: canvas, menú, HUD, pausa, instrucciones, nivel completado, fin y resultados.
3. Preparar assets de jugador, cuatro enemigos, casillas, bombas, explosiones, salida, power-ups, música y efectos.
4. Generar matriz del mapa con MazeGenerator, proteger el inicio, elegir posiciones de enemigos y ocultar la salida.
5. Implementar Player: lectura de teclas, movimiento con deltaTime, colisión, animación y habilidades temporales.
6. Implementar Bomb: temporización, propagación en cruz, destrucción de bloques y detonaciones encadenadas.
7. Añadir RandomEnemy, BFSEnemy, AStarEnemy y PredictiveEnemy con posiciones iniciales y puntuación.
8. Añadir mejoras, generación tras destruir bloques, límites, iconos, aura e inventario.
9. Integrar estados, dificultad, HUD, depuración, estilos espaciales y audio.
10. Integrar endpoints de Node.js, pantalla de resultados y comprobaciones de funcionamiento.

Estas etapas describen la secuencia lógica que permite reproducir el desarrollo a partir del código final. No representan fechas ni atribuciones individuales verificadas en Git.

4. Sistemas principales

4.1 Mapa procedural

MazeGenerator produce una matriz de 11×19 casillas. Los valores son 0 = suelo, 1 = muro indestructible y 2 = bloque destructible. El borde y las intersecciones pares forman muros fijos. Se protegen la salida inicial del jugador y una vía alrededor de cada enemigo.

Regla	Implementación
Bloques	Probabilidad $\min(0.34 + (\text{nivel} - 1) \times 0.025, 0.64)$.
Salida	Se elige lejos del inicio y queda oculta bajo un bloque.
Inicio	Jugador en columna 1, fila 1; casillas próximas despejadas.
Enemigos	Posiciones mezcladas, distancia Manhattan mínima 6 del jugador.
Tamaño gráfico	$19 \times 48 = 912$ px de ancho; $11 \times 48 = 528$ px de alto.

4.2 Jugador y colisiones

Player integra movimiento continuo por deltaTime, normalización diagonal, dirección/animación y detección por cuatro puntos con margen interno. La colisión se resuelve por eje para permitir deslizamiento. Las bombas dejan salir al jugador que las coloca y se vuelven sólidas después de abandonar su casilla.

4.3 Bombas y cadenas

Una bomba pasa de armed a exploding y finalmente finished. La mecha base es de 3000 ms y la explosión dura 600 ms. El algoritmo recorre arriba, abajo, izquierda y derecha hasta range; un muro detiene la propagación y un bloque recibe daño, se elimina y corta ese brazo. Una explosión puede forzar otra bomba y ampliar la cola de reacción; el tamaño máximo se conserva como estadística.

4.4 Inteligencia artificial

Clase	Algoritmo	Detalle de implementación
RandomEnemy	Selección aleatoria	Decide un vecino transitable y se desplaza casilla por casilla.
BFSEnemy	Breadth-First Search	Cola FIFO, conjunto visited y mapa parents; garantiza menor número de pasos en costos uniformes.
AStarEnemy	A*	openList, closedList, gScore y fScore; heurística Manhattan.
PredictiveEnemy	Predicción	Proyecta la dirección/posición futura del jugador para anticipar su ruta.

Las rutas se recalculan al terminar cada movimiento de casilla. En Debug, currentPath se representa sobre el Canvas, lo que permite verificar BFS y A*.

4.5 Power-ups

Tipo	Cambio de estado	Máximo / duración
bomb	maximumBombs += 1	5
range	bombRange += 1	6
speed	player.speed += 20	240 px/s
life	lives += 1	5
shield	player.addShield()	3 cargas
phase	player.activatePhase(8000)	8 s
remote	player.addRemoteDetonation()	5 cargas
special	player.addSpecialBomb()	5 cargas

Cada bloque destruido, excepto el que oculta la salida, tiene 35 % de probabilidad de generar una mejora. El inventario se actualiza al recoger o consumir cargas.

4.6 Progresión

Rango	Enemigos	Tiempo	Velocidad
Nivel 1	6	120 s	Base
Niveles 2-13	7 a 18	-10 s por nivel, mínimo 60	Base
Nivel 14 en adelante	18	60 s	+10 % por nivel desde el 14; máximo 250 %

Completar un nivel añade $500 \times$ nivel puntos. Los enemigos otorgan una puntuación asociada a su clase. Si el tiempo llega a cero se pierde una vida, el jugador vuelve al inicio y el tiempo del nivel se restablece.

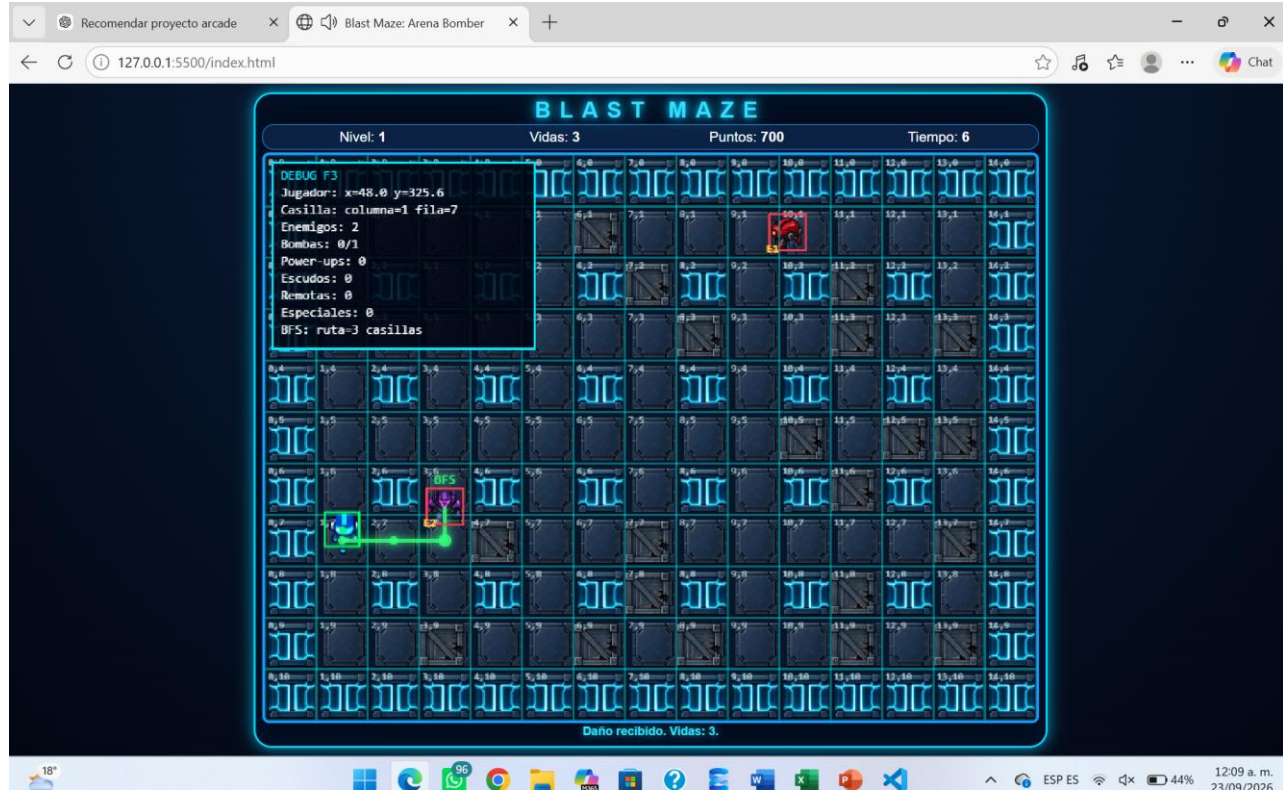
4.7 Puntuación y animación

main.js acumula puntos al eliminar enemigos con una explosión; el enemigo aleatorio recibe 100, BFS 200 y A* 300 puntos en los archivos disponibles. El valor del enemigo predictivo procede de su clase y requiere cotejo con enemyPredictive.js. La bonificación

de nivel es $500 \times \text{nivel}$. Los sprites se representan con `drawImage`: el cuadro de origen depende de `frameX`, `frameY` y dimensiones de la hoja; `deltaTime` actualiza las animaciones y el movimiento. Los efectos de explosión, aura y fase se dibujan en el contexto del Canvas.

4.8 Depuración

F3 activa cuadrícula y coordenadas, hitboxes, posiciones, conteos de entidades, inventario técnico y rutas de BFS/A*. Este modo permite comprobar criterios de búsqueda y colisión sin modificar el estado persistente.



Superposición de depuración con ruta BFS.

5. Persistencia y API

`server/server.js` usa únicamente módulos nativos de Node.js. Crea automáticamente `server/data/scores.json`, habilita CORS para desarrollo local, limita el cuerpo recibido a 100 000 caracteres y conserva las 1000 partidas más recientes.

Método y ruta	Respuesta	Uso
GET /api/health	200 · {status, message}	Comprobar disponibilidad.
GET /api/scores	200 · arreglo ordenado por puntuación	Poblar pantalla de resultados.
POST /api/scores	201 · mensaje y registro creado	Guardar fin de partida.
Cualquier otra ruta	404 · error	Control de rutas no existentes.

5.1 Esquema de registro

Campo	Tipo	Origen / validación
id	string	Timestamp + sufijo aleatorio.
puntuacion	entero ≥ 0	points.
nivelMaximo	entero ≥ 1	level.
enemigosEliminados	entero ≥ 0	Contador de derrotas.

Campo	Tipo	Origen / validación
duracionSegundos	entero ≥ 0	Fecha final – inicio.
mayorReaccionCadena	entero ≥ 0	Máximo chainSize.
fecha	ISO 8601	Generada por el servidor.

6. Instalación y ejecución

1. Instale Node.js 18 o superior.
2. Copie el proyecto conservando la estructura de carpetas y nombres de archivo.
3. Desde la raíz, ejecute `node server/server.js`.
4. Verifique `http://localhost:3000/api/health`.
5. Sirva el frontend con Live Server u otro servidor estático. No es recomendable abrir `index.html` mediante `file://`.
6. Abra el juego, inicie una partida y confirme en F12 > Network que `POST /api/scores` responde 201 al finalizar.

Puertos El backend utiliza 3000 y Live Server suele usar 5500. Si cambia PORT, actualice también `STATISTICS_API` en `main.js`.

7. Plan de pruebas

ID	Prueba	Resultado esperado
F-01	Iniciar una partida	Mapa 19 × 11, jugador, 6 enemigos y 120 s.
F-02	Mover contra muro/bloque	No atraviesa salvo fase activa para bloques.
F-03	Colocar bomba	Respeto límite y explota después de la mecha.
F-04	Encadenar bombas	Las alcanzadas explotan y se registra la cadena.
F-05	Derrotar enemigos y hallar salida	Portal se habilita solo sin enemigos.
F-06	Agotar tiempo	Resta vida, reinicia posición y tiempo.
F-07	Recoger cada power-up	Aplica efecto, límite e inventario correctos.
F-08	Pausa y retorno al menú	Congela tiempo; ambos botones responden.
F-09	F3	Muestra cuadrícula, hitboxes y rutas.
F-10	Fin de partida	POST guarda todos los campos.
F-11	Resultados	GET llena tarjetas y tabla.
F-12	Nivel 14	Mantiene 18 enemigos y aumenta velocidad 10 %.

7.1 Criterios de aceptación

- El laberinto se genera automáticamente y conserva rutas iniciales transitables.
- El jugador puede colocar bombas, destruir bloques y provocar cadenas.
- Existen al menos cuatro enemigos con algoritmos diferenciados.
- Los ocho power-ups modifican el estado de forma visible.
- La progresión continúa entre niveles sin un máximo funcional.
- El modo Debug evidencia cuadrícula, hitboxes, posiciones y rutas.
- El servidor persiste las cinco métricas solicitadas y la interfaz las consulta.

7.2 Evidencia disponible y pendiente

Las capturas disponibles muestran el escenario, la pausa, el modo Debug y una respuesta JSON de `/api/scores`. Una captura del endpoint muestra un registro de ejemplo con 1500 puntos, nivel 2, cuatro enemigos eliminados, 94 segundos y cadena máxima de 2. Es evidencia de esa ejecución anterior; no equivale a una prueba integral de la copia final.

Comprobación	Evidencia actual	Acción de cierre
Servidor	Captura de arranque y respuesta de /api/scores.	Repetir GET /api/health y guardar partida en la versión final.
Interfaz y juego	Capturas del mapa, Debug, pausa e inventario.	Probar en vivo menú, audio, progreso y final.
Enemigos y habilidades	Clases y reglas presentes en los fragmentos.	Probar en vivo cuatro clases y ocho mejoras.
Repositorio	No disponible en el material revisado.	Crear el repositorio y conservar commits reales de cada integrante.

8. Mantenimiento y extensión

Necesidad	Ubicación / parámetro
Cambiar tamaño de casilla	TILE_SIZE en main.js; revisar sprites y contenedor.
Cambiar cantidad de enemigos	INITIAL_ENEMIES y MAXIMUM_ENEMIES en map.js.
Cambiar dificultad	Probabilidad de bloques, temporizador y multiplicador en main.js/map.js.
Agregar power-up	Lista de types, PowerUp.frames/draw, applyPowerUp e inventario HTML.
Agregar enemigo	Nueva clase compatible, script antes de main.js y clase en enemyTypes.
Cambiar puerto	PORT en server.js y STATISTICS_API en main.js.
Cambiar assets	Mantener nombres/rutas o actualizar image.src y Audio.

8.1 Riesgos conocidos y recomendaciones

- scores.json es una persistencia local adecuada para el alcance académico, no para concurrencia o producción; migrar a base de datos si crece el uso.
- Access-Control-Allow-Origin: * facilita desarrollo; restringir el origen en despliegue real.
- La lista A* usa búsqueda lineal del mejor nodo; una cola de prioridad mejora el rendimiento en mapas grandes.
- Mantener un único bloque de variables DOM; duplicar declaraciones puede producir SyntaxError y detener todo main.js.
- Validar dimensiones y columnas de cada sprite sheet antes de cambiar imágenes para evitar recortes o distorsión.
- Agregar pruebas automatizadas para generación de mapas, rutas y normalización de la API si el proyecto continúa.

8.2 Catálogo de recursos gráficos

Recurso	Función y animación
player_sheet.png	Jugador; 5 columnas × 4 direcciones en Player.
enemy_random_sheet.png	Enemigo de movimiento aleatorio.
enemy_bfs_sheet.png	Enemigo perseguidor BFS.
enemy_astar_sheet.png	Enemigo perseguidor A*.
enemy_predictive_sheet.png	Enemigo predictivo.
tiles_sheet.png	Casillas de piso, paredes y bloques; 4 × 2 según main.js.
bomb_explosion_sheet.png	Bomba y explosión; 8 cuadros horizontales según Bomb.
exit_sheet.png	Portal de salida; 6 cuadros horizontales según main.js.
powerups_sheet.png	Diez cuadros horizontales; iconos de ocho mejoras.
player_states_sheet.png	Variantes visuales del jugador; comprobar uso en la copia final.
effects_sheet.png	Efectos visuales adicionales; comprobar uso en la copia final.
powerup_vida_corazon.png	Icono individual de corazón.
powerup_velocidad_bota_alado.png	Icono individual de calzado alado.
powerup_alcance_mira.png	Icono individual para alcance.
powerups_nuevos_sheet.png	Hoja de los tres iconos personalizados.
space_frame.png	Marco y paneles de la interfaz espacial.



Hoja con diseños de vida, velocidad y alcance.

No se utilizan jefes ni proyectiles separados en el diseño documentado: las bombas generan áreas de explosión. El suelo claro y sus colores por nivel se dibujan en Canvas; los bloques destructibles combinan sprite y oscurecimiento mediante dibujo.

8.3 Audio y derechos de uso

AudioManager referencia menu.mp3, game.mp3 y los efectos bomb-place.wav, explosion.wav, powerup.wav, damage.wav, enemy-defeated.wav, victory.wav y defeat.wav. Las fuentes originales, autores, enlaces, licencia y herramientas de edición de cada recurso no constan en los archivos disponibles. El equipo debe completar el registro de atribución antes de publicar en GitHub o presentar el proyecto; no se puede suponer que el material externo sea libre de uso.

Grupo	Dato que debe aportar el equipo
Sprites y marco	Quién lo creó, herramienta utilizada o URL de procedencia y licencia.
Música y efectos	URL, autor, licencia y si requieren atribución; comprobar que ningún archivo esté vacío.
Modificaciones	Indicar recortes, cambio de escala, recoloración y hojas nuevas.
Permisos	Conservar enlace o copia de la licencia de cada recurso externo.

9. Diagnóstico técnico

Síntoma	Revisión
Canvas vacío	Primera excepción en consola, IDs HTML nulos, orden de scripts y carga de map.js.
Cannot read properties of null	El ID esperado por main.js no existe o el script se ejecuta antes del DOM.
Identifier already declared	Se pegó dos veces una constante o bloque de código.
Sprites recortados	spriteColumns/frame no coincide con la hoja o el sourceWidth es incorrecto.
No guarda resultados	Backend detenido, CORS/puerto o JSON sin permisos de escritura.
Audio 0:00	Archivo de 0 bytes, ruta incorrecta o codificación no compatible.

9.1 Problemas identificados y solución

Problema y evidencia	Solución implementada o necesaria
Archivo MP3 de 0 bytes en una captura anterior.	Sustituir por audio reproducible y comprobar tamaño y reproducción.
Canvas vacío y excepciones de null/declaración duplicada.	Corregir IDs de elementos y consolidar declaraciones; revisar consola F12.
Escala y recorte de power-ups en capturas previas.	Mantener relación de aspecto, ajustar región visible e incrementar tamaño.
Necesidad de volver al menú desde pausa.	Se incluyó botón y controlador returnToMainMenu en la versión de código revisada.
Desajuste actual: index.html usa botonAudio; audio.js busca botonSonido.	Cambiar ambas búsquedas en audio.js a botonAudio y probar clic/tecla M.
Tres nombres en créditos frente a consigna de equipos de dos.	Confirmar con el docente autorización de tres integrantes.

9.2 Conclusiones y siguientes comprobaciones

El diseño separa mapa, entidades y servidor, y utiliza Canvas y tecnologías web autorizadas. La generación del laberinto, las rutas BFS/A*, las explosiones y la persistencia hacen posible una demostración técnica del proyecto. Antes de la entrega se deben verificar en la carpeta real los archivos faltantes en este entorno, las licencias de recursos, el clic del botón de audio y una ejecución completa con servidor. El historial de Git debe representar contribuciones reales; una carga única de la versión final no demuestra las etapas anteriores.

Cierre. La versión documentada satisface la generación procedural, combate con bombas, cadenas, cuatro estrategias de enemigo, progresión indefinida, power-ups, depuración y persistencia de estadísticas. La separación por clases permite ampliar comportamientos sin sustituir el núcleo del juego.

Créditos del proyecto: David Domínguez, Yenni Lopez y Samuel Pantaleon.